

Building a Highly Available and Scalable Web Infrastructure Using AWS Application Load Balancer and Auto Scaling Groups

By; Danuja

Introduction

Modern cloud applications must be designed to handle unpredictable traffic, maintain high performance, and remain available even during system failures. AWS enables this through two powerful services: **Application Load Balancer (ALB)** and **Auto Scaling Groups (ASG)**. Together, they form a resilient and self-healing architecture capable of adapting dynamically to demand.

The **Application Load Balancer** intelligently distributes incoming user traffic across multiple EC2 instances based on application-layer rules, ensuring optimal performance, fault tolerance, and seamless routing. Meanwhile, the **Auto Scaling Group** automatically adjusts the number of running EC2 instances—scaling out during peak demand and scaling in when traffic is low—lowering operational costs without compromising the user experience.

By integrating ALB with ASG, applications gain the ability to self-recover from failures, minimize downtime, and maintain consistent performance under varying workloads. This architecture is widely used in production environments to deliver **high-availability**, **fault-tolerance**, and **elastic scalability**—key characteristics of modern cloud-native applications.

Load Balancer Layer Best for

ALB (Application Load Balancer) Layer 7 Apps, microservices, HTTP/HTTPS traffic

NLB (Network Load Balancer) Layer 4 High performance, TCP/UDP traffic

CLB (Classic Load Balancer) Layer 4 & 7 Legacy workloads (old)

Scaling Type	Action	Best Use Case
Manual Scaling	User manually adjusts instance count	Testing, cost control, temporary scaling
Scheduled Scaling	Scales based on time (example: peak business hours)	Predictable workloads (e-commerce sales, working hours)
Dynamic / Policy-Based Scaling	Scales automatically based on metrics (CPU, requests, latency)	Unpredictable or fluctuating workloads
Predictive Scaling (AI-based)	Machine-learning forecasts demand and scales ahead	Enterprise workloads with repeating traffic patterns

Step-by-Step: Auto Scaling + ALB + EC2 (NGINX)

This defines how every EC2 in the Auto Scaling Group will look.

- Go to **EC2** → **Launch Templates** → **Create template**
- Name: `nginx-template`
- AMI: Amazon Linux 2023
- Instance Type: `t2.micro` or `t3.micro`
- Key pair: select if needed
- Networking: choose correct VPC + subnet (any public subnet)
- **Security group must allow:**
 - HTTP → port **80** → `0.0.0.0/0`
 - SSH → port **22** → your IP (optional)

```
#!/bin/bash
```

```
dnf update -y
```

```
dnf install -y nginx
```

```
systemctl enable nginx
```

```
systemctl start nginx
```

```
cat <<EOF >/usr/share/nginx/html/index.html
```

```
<h1 style="color: green;text-align:center;margin-top:50px;">
```

```
Hello from Auto Scaled EC2 Instance 🚀
```

```
</h1>
```

```
<p style="text-align:center;">Served by: $(curl -s http://169.254.169.254/latest/meta-data/local-ipv4)</p>
```

```
EOF
```

Launch Templates (1/1) [Info](#)



Actions

Create launch template

Search

< 1 > ⚙

☒	Launch Template ID	Launch Template Name	Default Version	Latest Version	Create
☒	lt-04ea56137884bfc8d	AutoScaling-template-1	1	1	2025-1

AutoScaling-template-1 (lt-04ea56137884bfc8d)



Launch template details

Actions

Delete template

Launch template ID

lt-04ea56137884bfc8d

Launch template name

AutoScaling-template-1

Default version

1

Owner

arn:aws:iam::875707075265:user/danuja-ncpl-practice

Details

Versions

Template tags

Launch template version details

Actions

Delete template version

Version

1 (Default)

Description

-

Date created

2025-12-06T08:28:39.000Z

Created by

arn:aws:iam::875707075265:user/danuja-ncpl-practice

Create two EC2 instance using launch template

go to Launch Templates

- Open **AWS Management Console**
- Search **EC2**
- On the left menu, click **Launch Templates**
- Click **Create Launch Template**

The screenshot displays the AWS Management Console interface for EC2 instances. At the top, there's a header with 'Instances (1/2)' and a search bar. Below the header, a table lists two running instances. The first instance, 'i-05828a7db980e932d', is selected. Below the table, the 'Details' tab is active for the selected instance, showing its ID, public IPv4 address (13.53.71.139), private IPv4 addresses (172.31.3.98), and its state as 'Running'.

Instances (1/2) Info Refresh Connect Instance state Actions Launch instances

All states

Instance state = running Clear filters < 1 > Settings

☐	Name	Instance ID	Instance state	Instance type	Status check	Alarm
☒		i-05828a7db980e932d	Running	t3.micro	3/3 checks passed	View
☐		i-0dfdc1edbb26e4fc	Running	t3.micro	3/3 checks passed	View

i-05828a7db980e932d Settings Dropdown

Details | Status and alarms | Monitoring | Security | Networking | Storage | Tags

▼ **Instance summary** Info

Instance ID i-05828a7db980e932d	Public IPv4 address 13.53.71.139 open address	Private IPv4 addresses 172.31.3.98
IPv6 address -	Instance state Running	Public DNS View

Server 1



Hello from Auto Scaled EC2 Instance

Served by:

Server 2



Hello from Auto Scaled EC2 Instance 02đŸš€

```
# Get IMDSv2 token TOKEN=AQAEANDSJRQo3tPVzDGm7ty13qyEu4rRxRauByOjgedN6ihSdrzLng== # Get EC2 private IP using the token INSTANCE_IP=
```

Create a Target Group

This tells the load balancer where to send traffic.

- Go to **EC2** → **Target Groups** → **Create**
- Type: **Instances**
- Protocol: **HTTP**
- Port: **80**
- VPC: same as your subnets
- **Health Check Path:** /
- Create

Load-balancer-targetGroup-1

Load-balancer-targetGroup-1

Actions

Details

arn:aws:elasticloadbalancing:eu-north-1:875707075265:targetgroup/Load-balancer-targetGroup-1/0c204c0bba361dae

Target type Instance	Protocol : Port HTTP: 80	Protocol version HTTP1	VPC vpc-0e32170621a153b4e
IP address type IPv4	Load balancer None associated		

2
Total targets

0
Healthy

0
Unhealthy

2
Unused

0
Initial

0
Draining

Distribution of targets by Availability Zone (AZ)

Select values in this table to see corresponding filters applied to the Registered targets table below.

Targets

Monitoring

Health checks

Attributes

Tags

Registered targets (2)

Anomaly mitigation: Not applicable

Deregister

Register targets

Target groups route requests to individual registered targets using the protocol and port number specified. Health checks are performed on all registered targets according to the target group's health check settings. Anomaly detection is automatically applied to HTTP/HTTPS target groups with at least 3 healthy targets.

Filter targets

☐	Instance ID	Name	Port	Zone	Health status	Health statu
☐	i-05828a7db980e932d		80	eu-north-1c (e...	Unused	Target group
☐	i-0dfdc1edbb26e4fc		80	eu-north-1c (e...	Unused	Target group

Create an Application Load Balancer (ALB)

- Go to **EC2** → **Load Balancers** → **Create Load Balancer**
- Type: **Application Load Balancer**
- Name: `nginx-alb`
- Scheme: Internet-facing
- IP type: IPv4

Network Mapping:

- Choose **at least 2 public subnets in different AZs**

Security Group:

- Must allow:
 - ✓ HTTP (80) from anywhere

Listeners:

- Listener: HTTP:80
- **Forward to:** your **Target Group** created in step 2

Create the ALB.

EC2 > Load balancers

icated Hosts

acity Reservations

acity Manager New

ages

Is

I Catalog

istic Block Store

umes

ipshots

ycle Manager

etwork & Security

urity Groups

stic IPs

ement Groups

r Pairs

etwork Interfaces

ad Balancing

id Balancers

get Groups

st Stores

to Scaling

o Scaling Groups

tings

Load balancers (1/1) What's new?

Actions

Create load balancer

Elastic Load Balancing scales your load balancer capacity automatically in response to changes in incoming traffic.

Filter load balancers

< 1 >

Name

State

Type

Scheme

IP address type

Loadbalancer-1

Active

application

Internet-facing

IPv4

Load balancer: Loadbalancer-1

< Network mapping Resource map Security Monitoring Integrations Atti >

Security groups (1) Edit

A security group is a set of firewall rules that control the traffic to your load balancer.

Security Group ID

Name

Description

[sg-Od96a2d27bedeaab1](#)

default

default VPC security group

loadbalancer-1-103213507.eu-north-1.elb.amazonaws.com

WhatsApp Gmail YouTube Account Settings Mail - Outlook Timetable Dashboard | Moodle

loadbalancer-1-103213507.eu-north-1.elb.amazonaws.com

WhatsApp Gmail YouTube Account Settings Mail - Outlook Timetable Dashboard | Moodle

Hello from Auto Scaled EC2 Instance 02dYš€

Get IMDSv2 token TOKEN=AQAEANDSIRQo3PPvzDGm7yI3yEia4RtRanByOjgedN6ihSdrlZag== # Get EC2 private IP using the token INSTANCE_IP=

Hello from Auto Scaled EC2 Instance dYš€

Served by:

Traffic is successfully distributed between both servers in a round-robin manner.

Create the Auto Scaling Group (ASG)

- Go to **EC2** → **Auto Scaling Groups** → **Create**
- Name: `nginx-asg`
- Select the Launch Template you created earlier
- VPC: choose same VPC
- Select **2+ subnets** (multi-AZ)
- Continue

Target Group Registration:

- Choose **Attach to an existing load balancer**
- Select your ALB
- Select your Target Group

Scaling Configuration:

- Desired capacity: **2**
- Minimum: **1**
- Maximum: **3** (or more if you want scaling demo)

The screenshot displays the AWS Management Console interface for configuring a Target Group. At the top, the 'Target groups (1/1)' section is visible, with a search bar and a table listing the target groups. The table has columns for Name, ARN, Port, Protocol, and Target. One target group is listed: 'Load-balancer-targetGr...' with ARN 'arn:aws:elasticloadbalancing...', Port '80', Protocol 'HTTP', and Target 'Instar'.

Below the table, the 'Target group: Load-balancer-targetGroup-1' details are shown. The 'Details' tab is selected, displaying the following information:

- Target type:** Instance
- Protocol : Port:** HTTP: 80
- Protocol version:** HTTP1
- VPC:** [vpc-0e32170621a153b4e](#)
- IP address type:** IPv4
- Load balancer:** [Loadbalancer-1](#)

Expected Result

- ✓ EC2 instances auto installed with NGINX
- ✓ ALB distributes traffic across multiple instances
- ✓ ASG replaces unhealthy EC2 and scales based on load
- ✓ Architecture is highly available across multiple AZs